Que Skills de Teste as Agent Skills Ensinam? Um Perfil Preliminar via Practical Test Pyramid no GitSkills

Pamella Soares
Universidade Estadual do Ceará (UECE)
Fortaleza, Brasil
pamella.soares@aluno.uece.br

***Resumo*—Agent skills (`SKILL.md`) instruem coding agents em linguagem natural, sem compilador que valide a orientação de teste. Neste *short paper* do MSR 2027 Mining Challenge, perguntamos quais skills de teste são evidenciadas em skills do domínio blockchain no dataset GitSkills. Operacionalizamos a *Practical Test Pyramid* (Vocke/Fowler) como glossário de níveis de teste e de skills relacionadas (Unitário, Integração, Contrato, UI, E2E, Aceitação, Exploratório, Automação). A partir de 89.649 skills com vocabulário blockchain, filtramos 17.847 com orientação lexical a teste: Unitário (30,0%) e Integração (23,6%) dominam; E2E (14,3%) e Aceitação (2,8%) são menos frequentes; sinais Web3 como fuzz/invariant (9,9%) e fork (2,8%) aparecem aquém de coverage/regression. Discutimos os impactos desses desequilíbrios para o desenvolvimento de sistemas baseados em blockchain (falsos verdes, superfície DeFi, pontes, carteiras e governança de agentes). Um piloto LLM (checklist com evidência verbatim) e uma amostra cega estratificada (n=32) sustentam o protocolo. Não medimos proficiência—apenas *skill evidenciada* no texto.**

***Index Terms*—Mining Software Repositories, GitSkills, agent skills, test pyramid, blockchain, testing skills**

I. Introdução

Agent skills são pastas com um `SKILL.md` que orienta um agente de IA sobre como executar tarefas [1]. Diferente de código, o conteúdo é predominantemente linguagem natural: não há type checker que diga se a orientação de teste é completa ou adequada ao domínio. Em sistemas baseados em blockchain e DApps, falhas de verificação têm custo alto—fundos imobilizados, exploits, estado on-chain difícil de reverter e risco reputacional para protocolos.

A pilha prática de verificação Web3 não se reduz a "rodar unit tests": inclui integração entre aplicação, contrato e rede; testes de ponta a ponta; propriedades/invariantes; fuzzing; forks de mainnet; e critérios de oráculo (reverts, eventos, saldo, autorização). Quando um coding agent é guiado por skills incompletas, o risco não é apenas acadêmico: o agente pode gerar ou modificar contratos e scripts de deploy com uma suíte que parece suficiente e ainda assim omite níveis críticos.

O MSR 2027 Mining Challenge disponibiliza o GitSkills [1] e incentiva taxonomias de propósito e métricas para instruções em linguagem natural. Este artigo responde, de forma preliminar:

- **RQ1 (quantitativa):** Com que frequência skills blockchain mencionam cada **nível de teste** da pirâmide prática (e sinais correlatos Web3)?
- **RQ2 (qualitativa/piloto):** Quais *testing skills* o texto evidencia quando anotado via checklist da Practical Test Pyramid?
- **Impacto:** O que esses padrões implicam para quem desenvolve, audita ou opera sistemas blockchain com auxílio de agentes?

a) Escopo.: Não propomos taxonomia ISTQB/ISO completa; adotamos o vocabulário operacional de Vocke/Fowler [2], que alerta para ambiguidade de nomes de níveis. Classificamos *skill evidenced / skill instructed*, não proficiência do autor. Não afirmamos causalidade entre skill e defeito em produção; discutimos *riscos plausíveis* à luz da literatura prática de verificação on-chain.

II. Referência: Practical Test Pyramid

A pirâmide clássica (Cohn) agrupa testes por granularidade: muitos testes de baixo nível, poucos de alto nível, buscando feedback rápido e custo controlado. Vocke [2] discute Unit, Integration, Contract, UI, End-to-End e Acceptance, além de Exploratory e automação/CI como preocupações transversais. Acceptance é **ortogonal** à pirâmide (não forçamos "aceitação = topo"). Para mineração com LLM, isso motiva um checklist curto (Q1–Q8) em vez de uma taxonomia de técnicas/tipos.

No domínio blockchain, os mesmos níveis ganham significados concretos: unitário sobre funções de contrato ou bibliotecas; integração entre serviços off-chain e contratos; contrato no sentido de interfaces consumidor–provedor (p.ex., oráculos, bridges); UI/API de carteiras e front-ends; E2E atravessando usuário→API→contrato→cadeia; aceitação alinhada a requisitos de negócio/protocolo.

III. Metodologia

A. Visão geral (pipeline)

1) Partir do snapshot GitSkills (coleta jul/2026; linhagem `agent_skills_mandatory_only`).
2) Restringir a skills com $\geq$1 termo blockchain (*mandatory terms*).
3) Filtrar lexicalmente orientação a teste / níveis de teste (pool candidato).

4) Agregar prevalência por nível (RQ1), exclusiveidade e coocorrência.
5) Amostrar piloto e aplicar checklist Pyramid via LLM (RQ2).
6) Preparar amostra cega estratificada para validação humana.
7) Interpretar resultados sob a ótica de impacto em desenvolvimento blockchain.

B. Dados

- **Dataset:** GitSkills [1], versão alinhada ao desafio (jul/2026).
- **Base analítica:** `agent_skills_mandatory_only.db` — 89.649 skills distintas (`dedup_primary=1`) com $\geq$1 termo blockchain.
- **Unidade:** um artefato `SKILL.md` (`name`, `description`, `content`).
- Toda a trilha parte de `mandatory_only` (não da base `blockchain_filtered` alternativa).

C. Filtro lexical por nível de teste (RQ1)

- Vocabulário em frases compostas (evita o termo isolado "test"); busca case-insensitive em conteúdo+nome+descrição.
- Níveis nucleares: *unitário*, *integração*, *ponta a ponta (E2E)*, *aceitação*.
- Sinais adicionais (não substituem o nível): coverage, regression, fork, fuzz/property, mutation, gas/performance, etc.
- Entrada no pool: $\geq$1 termo em qualquer grupo $\Rightarrow$ **17.847** skills ($\approx$19,9% da base blockchain lexical).
- Multi-rótulo lexical permitido; reportamos prevalência "qualquer menção" e contagens exclusivas.
- Artefatos: listas de termos, pool JSON, scripts de filtragem e plots.

D. Checklist Pyramid e piloto LLM (RQ2)

- Instrumento Q1–Q8: Unitário, Integração, Contrato, UI, E2E, Aceitação, Exploratório, Automação.
- Respostas: `YES`/`NO`/`UNCERTAIN`; todo `YES` exige quote verbatim.
- A LLM responde o checklist; regras em código derivam `skills_evidenced` só com quote validável.
- Princípios: multi-rótulo com evidência independente; exploratory só com menção explícita; não avaliar qualidade.
- Piloto: 6 skills (3 exemplos de pirâmide + 3 do pool); 5 classificadas como orientação a teste.

E. Validação humana (em andamento)

- Amostra cega: **8** skills por nível lexical exclusivo $\times$ 4 níveis $= n$=32 (seed 20270402).
- Revisores não veem o estrato lexical; protocolo em `PROTOCOL_MANUAL_REVIEW.md`.
- κ/α após dupla anotação entram na versão estendida; aqui reportamos o desenho.

Tabela I
PREVALÊNCIA LEXICAL NO POOL DE ORIENTAÇÃO A TESTE.

Nível / sinal	Skills	% pool	Exclusivo
Unitário	5.348	30,0	2.240
Integração	4.215	23,6	1.302
E2E	2.552	14,3	751
Aceitação	496	2,8	326
Coverage	5.344	29,9	—
Regression	4.748	26,6	—
Fuzz / property	1.772	9,9	—
Fork	497	2,8	—

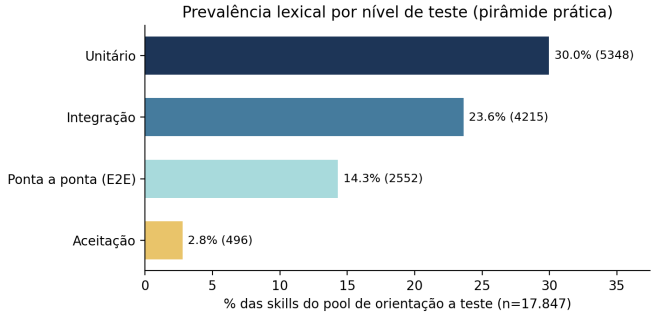


Figura 1. Prevalência lexical por nível de teste (n=17.847).

IV. RESULTADOS

A. RQ1 — Prevalência por nível de teste

No pool (n=17.847), a prevalência lexical (não exclusiva) é resumida na Tabela I e nas Figuras 1–2.

Além disso: 3.295 skills (18,5%) combinam $\geq$2 níveis nucleares (coocorrência mais frequente: unitário+integração, 2.739); 9.933 skills (55,7%) entram no pool por sinais de teste *sem* mencionar os quatro níveis nucleares—muitas via coverage/regression ou outros tipos. Isso indica um corpus em que "falar de teste" frequentemente não especifica *qual* nível da pirâmide praticar.

B. RQ2 — Testing Skill Profile (piloto)

Nas 5 skills com orientação a teste no piloto LLM, Automação apareceu em 4/5; Unitário, Integração e Contrato em 3/5; UI e E2E em 2/5; Aceitação e Exploratório em 0/5 (Figura 3). Exemplos qualitativos: uma skill de desenvolvimento Solidity enfatizou unitário+cheatcodes/asserções; uma de pentest de DApps evidenciou integração backend–blockchain, UI/XSS em carteiras e E2E de fluxos; skills com fuzz/fork no texto lexical tenderam a evidenciar automação e técnicas de propriedade—mas Aceitação e Exploratório permaneceram ausentes. Quotes não verbatim foram descartadas automaticamente.

V. IMPACTOS PARA O DESENVOLVIMENTO DE SISTEMAS BLOCKCHAIN

Os números não medem defeitos em produção; medem o que as *instruções* disponíveis aos agentes enfatizam. Em um ecossistema onde agents passam a escrever, refatorar

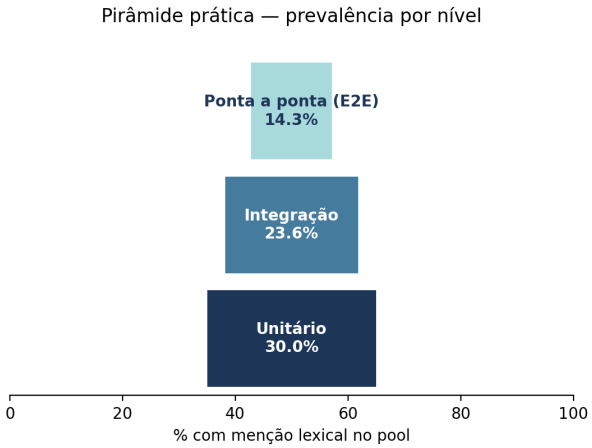


Figura 2. Pirâmide prática: Unitário, Integração e E2E (% do pool).

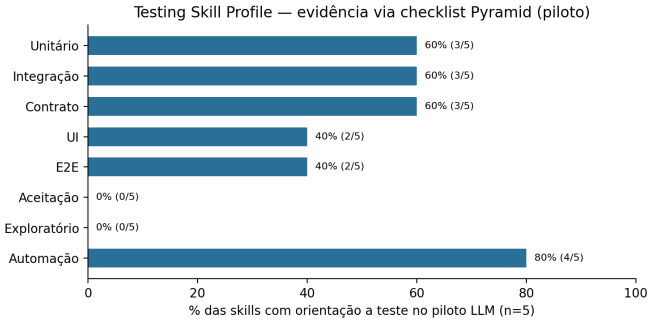


Figura 3. Cobertura de skills evidenciadas no piloto LLM.

e "validar" contratos, esse viés de instrução torna-se parte do risco de engenharia. Organizamos os impactos por nível omitido ou sub-representado.

A. *Unitário frequente, porém insuficiente sozinho*

Unitário é o nível mais citado (30,0%). Para bibliotecas Solidity/Vyper e lógica pura, isso é positivo: feedback rápido, asserts de revert/evento, isolamento. O impacto negativo aparece quando a skill (e, por extensão, o agente) trata unitário como *definição* de "bem testado". Em DeFi, muitos exploits exploram interações, ordem de mensagens, oráculos e estados compostos—cenários que unitários com mocks agressivos podem mascarar. **Impacto prático:** times que confiam em agents guiados por skills unit-cêntricas podem acelerar entrega com cobertura aparente alta e ainda assim deixar aberta a superfície de integração.

B. *Integração presente, mas assimétrica*

Integração (23,6%) é o segundo nível nuclear. Isso é encorajador para pontes app↔contrato e sync off-chain/on-chain. Contudo, coocorrência forte com unitário (2.739 skills) sugere que "integração" muitas vezes aparece no mesmo pacote retórico genérico, sem garantir profundidade (p.ex., testes com nó real, anvil/hardhat network, ou dependências externas verdadeiras). **Impacto:** em carteiras, indexadores e backends de DApp, falhas de integração geram transações mal formadas, nonces inconsistentes e estados divergentes—exatamente o tipo de defeito que usuários finais percebem como "a blockchain falhou", embora a falha seja de composição de sistemas.

C. *E2E relativamente raro (14,3%)*

E2E exercita o fluxo completo via interface externa. Em blockchain, isso frequentemente significa: carteira→front→RPC→contrato→eventos/indexação. A menor prevalência implica que agents podem ser pouco instruídos a validar jornadas reais (mint, swap, stake, bridge) sob condições de rede. **Impacto:** regressões de UX e de caminho feliz/infeliz em produção; custo operacional de suporte; em pior caso, usuários assinando fluxos parcialmente quebrados. Skills E2E genéricas (p.ex., Playwright Web2) sem oráculos on-chain ainda podem dar falso conforto.

D. *Aceitação quase ausente (2,8%)*

Aceitação—cenários de negócio/stakeholder—é rara no léxico das skills. Protocolos blockchain codificam regras econômicas e de governança: limites de mint, papéis, upgrades, pausas, taxa. Sem aceitação explícita, o agente pode "passar nos testes" sem verificar se a *feature* atende à especificação de negócio ou ao whitepaper operacional. **Impacto:** desalinhamento entre requisito econômico e implementação; auditorias tardias descobrem que o comportamento testado não é o comportamento desejado pelo protocolo.

E. *Fuzz/invariant e fork sub-representados*

Fuzz/property (9,9%) e fork (2,8%) ficam bem abaixo de coverage (29,9%) e regression (26,6%). Na prática BOSE, fuzz/invariant e forks de mainnet são ferramentas centrais para estados raros, propriedades DeFi e reprodução de condições reais de liquidez/slots. Coverage/regression, embora úteis, não substituem busca adversária de estado. **Impacto:** agents otimizando para "coverage report verde" em vez de propriedades quebráveis; menor chance de detectar edge cases que auditores e hunters buscam; risco particularmente alto em upgrades e em composabilidade DeFi.

F. *Automação sem pirâmide completa*

No piloto, Automação foi a skill mais frequente entre as evidenciadas. Automatizar uma suíte enviesada para a base da pirâmide *acelera* o viés: CI passa rápido, o agente recebe reforço positivo, e níveis altos/raros continuam omitidos. **Impacto:** pipelines que institucionalizam o falso verde; governança frágil de "definition of done" para PRs gerados por IA.

G. *Síntese para papéis do ecossistema*

- **Times de protocolo/DeFi:** tratar skills unit-cêntricas como incompletas por padrão; exigir invariantes, fuzz e cenários de integração/composição no Definition of Done do agente.
- **Carteiras e front-ends:** priorizar skills com UI/API + E2E que incluam assinatura, cadeia e falhas de RPC—não apenas testes de componente React.

- **Bridges e oráculos:** enfatizar contract/integration testing e forks; o custo de falha de interface é sistêmico.
- **Auditores e securidade:** usar o gap (aceitação/E2E/fuzz/fork) como checklist de revisão de PRs assistidos por agentes.
- **Autores de skills / marketplaces:** publicar skills com perfil de pirâmide explícito (quais níveis instrui) para reduzir overtrust.
- **Pesquisadores MSR:** o *Testing Skill Profile* oferece métrica de qualidade de instrução alinhada ao challenge (propósito + evidência).

VI. Ameaças à validade

- **Construto:** menção lexical $\neq$ orientação acionável; mitigamos parcialmente com checklist+quote na RQ2.
- **Interna:** overclassification LLM; quotes inválidas descartadas; piloto pequeno (n=6).
- **Externa:** base `mandatory_only` tem alto recall (ruído de termos ambíguos); skills espelham templates, não necessariamente prática de protocolos top-tier.
- **Conclusão:** impactos são *riscos plausíveis*, não efeitos medidos em mainnet.
- Ambiguidade de nomes de níveis (reconhecida por Vocke/Fowler); amostra humana ainda sem κ publicado.

VII. Implicações Éticas

Analisamos artefatos públicos do GitSkills. Não reidentificamos autores além dos metadados do dataset. A amostra cega reduz viés de confirmação dos revisores. Não executamos skills nem comandos nelas prescritos. Discussões de impacto não devem ser lidas como acusação a repositórios específicos.

VIII. Conclusão

Nas agent skills blockchain do GitSkills, a orientação a teste concentra-se nos níveis unitário e de integração e em proxies como coverage/regression; E2E, aceitação, fuzz/property e fork são comparativamente raros. Para o desenvolvimento de sistemas blockchain, isso significa um risco estrutural de *overtrust* em agentes: acelerar implementação com feedback automatizado na base da pirâmide, enquanto propriedades econômicas, jornadas ponta a ponta e condições de mainnet permanecem pouco instruídas. O checklist da Practical Test Pyramid oferece um *Testing Skill Profile* auditável—base para validação humana (n=32) e para melhorar catálogos de skills voltados a verificação on-chain.

Disponibilidade de Artefatos

Filtros, plots, piloto LLM, amostra cega e protocolo: `2027_short_paper_challenge/`. Dataset: GitSkills [1].

Referências

[1] G. Destefanis, D. Graziotin, M. Vaccargiu, and M. Ortu, "GitSkills: A dataset of agent skills on GitHub," in *Proceedings of the 24th International Conference on Mining Software Repositories*. Dublin, Ireland: IEEE, 2027, mining Challenge; preprint arXiv:2608.10906. [Online]. Available: https://arxiv.org/abs/2608.10906

[2] H. Vocke, "The practical test pyramid," Martin Fowler's blog, 2018, accessed: 2026-10-02. [Online]. Available: https://martinfowler.com/articles/practical-test-pyramid.html